

Cara Membuat SKILL.md untuk Claude Code

Panduan singkat bikin skill file biar Claude Code kamu lebih pintar dan konsisten ngerjain tugas spesifik.

Buat kamu yang nulis kode pakai Claude Code di terminal: skill files ini cara paling ampuh buat ngajarin Claude pola kerja kamu tanpa perlu ngulang instruksi tiap sesi.

■ Format file: **.md**

■ Lokasi: **.claude/skills/**

■ Auto-inject: **globs**

Apa Itu SKILL.md?

SKILL.md adalah file instruksi spesifik yang bisa kamu buat buat ngajarin Claude Code cara ngerjain tugas tertentu dengan gaya dan standar kamu.

Analogi:

Kayak .gitignore. Git cuma baca file .gitignore buat tau file mana yang diabaikan. Sama, Claude cuma baca file SKILL.md di folder .claude/skills/ buat tau standar kamu.

Kenapa Harus Pakai?

- **Konsisten.** Claude selalu pake standar yang kamu tentukan, nggak usah khawatir hasilnya random tiap sesi.
- **Hemat waktu.** Buat sekali, pake selamanya. Nggak perlu ngetik ulang instruksi yang sama.
- **Auto-inject.** Claude otomatis load skill yang cocok berdasarkan file yang lagi kamu buka (via pola **globs**).
- **Bisa dibagi.** Skill file bisa di-commit ke repo biar satu tim pake standar yang sama.

Format File SKILL.md

File SKILL.md pakai format **YAML frontmatter** di atas, isi bebas di bawah. Ini dia struktur dasarnya:

```
---
name: commit-message
description: "Use when generating git commit messages. Enforces Conventional Commits
format with scoped types: type(scope): description."
version: 1.0.0
author: kamu
license: MIT
metadata:
tags: [git, commit, conventional-commits, workflow]
---
```

Penjelasan Field:

Field	Fungsi
name	Nama skill. Format kebab-case.
description	Deskripsi singkat. Claude pake ini buat milih skill yang relevan.
version	Versi skill. Pake semver (1.0.0).
author	Pembuat skill.
license	Lisensi (MIT, Apache, dll).
metadata.tags	Tag buat kategori dan pencarian.

Isi File (Markdown)

Setelah frontmatter, kamu tinggal tulis instruksi dalam Markdown biasa. Bisa daftar aturan, contoh kode, atau langkah-langkah. Semakin spesifik, semakin baik.

Contoh isi skill TypeScript:

- Gunakan TypeScript strict mode di tsconfig.json
- Type hints wajib di semua function signature
- Gunakan interface, bukan type, untuk object shapes
- Prefer type inference daripada explicit type
- Gunakan enum untuk nilai tetap yang terbatas

Contoh Skill TypeScript

Skill ini auto-load setiap kali kamu buka file .ts atau .tsx:

```
---
name: typescript-convention
description: "TypeScript strict mode coding standards. Enforces type safety,
interfaces, and clean patterns."
version: 1.0.0
author: kamu
license: MIT
metadata:
tags: [typescript, coding-standard, type-safety]
---

# TypeScript Convention

- Strict mode WAJIB nyala di tsconfig.json.
- Semua function parameter pake type hint. Hindari any.
- Pake interface buat object shapes, type buat union/utility.
- Async function selalu pake Promise, bukan void return.
- Import type pake import type { ... } biar tree-shake aman.
- Error handling pake try-catch, bukan .catch() chain.
- Console.log cuma buat debugging, jangan commit.
```

Hasilnya:

Kamu bilang: "Bikin API service buat fetch user data"

Claude otomatis:

- Bikin interface User dengan type hints lengkap
- Async function dengan Promise return type
- Error handling pake try-catch
- Import type pake import type {}

Nggak perlu ngomong panjang lebar - skill file udah ngatur semuanya.

Tips & Best Practices

1. Mulai dari yang spesifik

Bikin skill per stack/framework. TypeScript sendiri, React sendiri, bukan "web-dev" yang general.

2. Gunakan globs dengan benar

`**/*.ts` auto-load pas buka TypeScript file. `**/*.{ts,tsx}` buat React+TS. Atur biar skill muncul di konteks yang tepat.

3. Tulis aturan konkret

Bukan "tuliskan kode yang baik" tapi "pake interface, bukan type. Type hints wajib."

4. Jangan panjang-panjang

Cukup 5-15 baris aturan. Skill yang kepanjangan bikin Claude bingung prioritasnya.

5. Commit ke repo tim

Taruh `.claude/skills/` di repo biar satu tim bisa pake standar yang sama.

Quick Start: 3 Langkah

- | | | |
|---|--------------------|--|
| 1 | Buat Folder | Buat folder <code>.claude/skills/</code> di root project kamu. |
| 2 | Buat File | Buat file <code>.claude/skills/typescript-convention.md</code> , isi frontmatter + aturan. |
| 3 | Mulai Pake | Buka file <code>.ts</code> dan mulai chat dengan Claude. Skill otomatis aktif. |

Siap bikin skill pertamamu?

Buka terminal, masuk ke project kamu, dan jalanin: `mkdir -p .claude/skills/`

Buat file `.md` pertama, isi 3-5 aturan, dan lihat sendiri bedanya.

Contoh Skill Siap Pakai

Tiga skill berikut bisa langsung kamu copy ke folder `.claude/skills/` dan mulai dipake. Tinggal save, restart Claude, dan skill otomatis aktif.

1. commit-message.md

```
---
name: commit-message
description: "Use when generating git commit messages. Enforces Conventional Commits format with scoped types:
type(scope): description."
version: 1.0.0
author: kamu
license: MIT
metadata:
tags: [git, commit, conventional-commits, workflow]
---

# Commit Message Convention
Tiap commit WAJIB pakai format Conventional Commits:
```

type(scope): deskripsi singkat (max 50 karakter)
- Baris opsional buat jelasin kenapa perubahan ini dilakukan
- Atau apa dampaknya ke codebase
```

Subject line pakai present tense, imperative mood ("add" bukan "added" atau "adds").

## Tipe
| Type | When |
| ----- | ----- |
| `feat` | Fitur baru buat user |
| `fix` | Bugfix buat user |
| `refactor` | Refactor -- no behavior change |
| `test` | Nambah atau ubah test |
| `docs` | Cuma dokumentasi |
| `chore` | Tooling, config, dependency |
| `style` | Formatting, whitespace, missing semicolons |
| `perf` | Performance improvement |

## Scope
Scope opsional -- nunjukin bagian codebase yang kena:
- `auth` -- login, register, session
- `api` -- endpoint handler
- `ui` -- component visual
- `db` -- migration, schema
- `config` -- env, settings
- `ci` -- pipeline, deploy config

## Contoh
```
feat(auth): add google oauth login
fix(api): handle timeout on slow response
refactor(ui): split navbar into components
test(db): add migration rollback test
chore: update dependencies
docs(api): add endpoint usage examples
```

## Pitfalls
- Jangan pake feat buat perubahan kecil yang gak kelihatan user -- pake chore
- Jangan pake fix kalo belum ada user yang ngalamin bug itu
- Jangan pake past tense di subject ("added", "fixed") -- pake "add", "fix"
- Kalau ragu antara dua tipe, pilih yang paling deket ke user impact

## Verification Checklist
- [ ] Subject max 50 karakter
- [ ] Format: type(scope): description
- [ ] Present tense, imperative mood
- [ ] Body (jika ada) jelasin WHY, bukan WHAT
```

2. code-review-focus.md

```
---
name: code-review-focus
description: "Use when reviewing pull requests or code changes. Enforces a structured review checklist: logic, security, performance, readability, and consistency."
version: 1.0.0
author: kamu
license: MIT
metadata:
tags: [code-review, pr, quality, best-practice]
---

# Code Review Focus Checklist

Setiap review WAJIB melewati 5 layer ini. Jangan cek cuman typo atau formatting.

## Layer 1 - Logic Correctness
- [ ] Apakah solusi ini benar untuk problem yang ada?
- [ ] Ada edge case yang gak ke-handle? (empty, null, unexpected input)
- [ ] Error handling proper? Ada try-catch atau fallback?
- [ ] State mutation aman? Gak ngubah data di luar scope?

## Layer 2 - Security
- [ ] User input di-validate / sanitize?
- [ ] Ada SQL injection risk? (query parameter, bukan concatenation)
- [ ] API endpoint ada rate limit / auth check?
- [ ] Secret / token hardcoded di code? (env variable dulu)

## Layer 3 - Performance
- [ ] Looping di dalam looping?  $O(n^2)$  bisa dihindari?
- [ ] Database query pake index? N+1 query problem?
- [ ] Bundle size / payload gak membengkak? (tree-shaking, code splitting)
- [ ] Caching bisa dipasang? Atau malah bikin stale data?

## Layer 4 - Readability
- [ ] Nama variable/function jelas maksudnya? Bukan x, temp, data
- [ ] Function terlalu panjang? (max ~30 lines per function)
- [ ] Comment jelasin "why" bukan "what" -- kode sendiri udah jelasin what
- [ ] DRY -- ada duplikasi yang bisa di-extract?

## Layer 5 - Consistency
- [ ] Coding style sesuai sama project convention (ESLint, Prettier, dll)
- [ ] Import urutnya rapi? (third-party dulu, internal setelahnya)
- [ ] Naming convention konsisten? (camelCase, PascalCase, CONSTANT_CASE)
- [ ] Dependency baru justified? Atau bisa pake yang udah ada?

## Pitfalls
- Jangan cuma fokus di formatting -- formatting itu kerjaan linter, bukan reviewer
- Jangan nitpick minor preferences (spasi vs tab) kalo udah ada tool yang handle
- Jangan approve cuma karena "code works" -- cek security & performance juga
- Kalau nemu hal bagus, bilang -- review bukan cuma nyari kesalahan

## Verification Checklist
- [ ] 5 layer dicek: Logic > Security > Performance > Readability > Consistency
- [ ] Komentar konstruktif (problem + saran solusi)
- [ ] Gak ada nitpick yang bisa di-handle linter
```

3. unit-test.md

```
---
name: unit-test
description: "Use when writing unit tests for Node.js / TypeScript projects. Enforces Vitest with describe/it
pattern, behavior-focused assertions, and clean test isolation."
version: 1.0.0
author: kamu
license: MIT
metadata:
tags: [test, vitest, nodejs, typescript, quality]
---

# Unit Test Convention
## Framework & Tools
- Runner: Vitest
- Assertion: expect() native Vitest
- Mocking: vi.mock() / vi.spyOn()
- Coverage target: >= 80% per modul
- Nama file: *.test.ts atau *.spec.ts, taruh di sebelah file yang di-test
## Test Structure - Arrange / Act / Assert
...

describe("NamaFunction / ModuleName", () => {
  describe("given [kondisi input / state]", () => {
    it("should [expected behavior / output]", () => {
      // arrange -- setup data, mock dependencies
      // act -- panggil function
      // assert -- cek hasilnya
    });
  });
});

## Aturan Wajib
1. Satu it() = satu behavior. Kalau butuh assert banyak, berarti perlu test terpisah.
2. Ganti describe("given ...") tiap ada kondisi input yang beda.
3. Jangan panggil API asli di unit test -- mock semua IO (database, HTTP, filesystem).
4. Nama test readable: describe("given user is logged in") -- bukan describe("case 1").
5. Jangan share state antar test tanpa cleanup -- pakai beforeEach buat reset.
## Pitfalls
- Jangan test internal implementation -- test behavior/output yang visible ke caller
- Jangan share variable antar test tanpa cleanup -- bisa contamination
- Mock return value yang realistis -- jangan asal return {} atau true
- Jangan pake beforeAll/afterAll kalo bisa pake beforeEach/afterEach
## Verification Checklist
- [ ] Format: describe > describe(given) > it(should)
- [ ] Satu it() = satu assertion utama
- [ ] Semua IO external di-mock
- [ ] Nama file: *.test.ts sampling file asli
- [ ] Run: npx vitest run -- all green
```

Download 3 skill ini dan mulai pake sekarang.

Copy file ke folder .claude/skills/ di project kamu. Claude akan otomatis load dan ngikutin standar ini.

Dibikin oleh @satriabahari